

STEP 4 — Submit + Approval + Rework

This is the **most important step** for your requirement.

STEP 4 — APPROVAL WORKFLOW, REWORK AND CYCLES

Implement the complete Approval Management decision workflow.

STRICT SCOPE:

Approval Management backend only.

=====

WORKFLOW

=====

Initial:

Draft

→ PendingApproval

Director decision:

PendingApproval

→ Approved

OR

PendingApproval

→ Rejected

OR

PendingApproval

→ ChangesRequested

Then:

ChangesRequested

→ Resubmitted

→ PendingApproval

=====

CYCLE RULE — CRITICAL

=====

Every request starts with:

CycleNo = 1

Example:

ReferenceNo:
APR-2026-000001

TaskId:
X

Cycle 1

When Director requests changes:

Cycle 1:
PendingApproval
→ ChangesRequested

When user resubmits:

Create Cycle 2.

BUT:

ReferenceNo remains:

APR-2026-000001

TaskId remains:

X

Only:

CycleNo = 2

If changes are requested again:

Cycle 2
→ ChangesRequested

Resubmit:

Cycle 3

Again:

Same ReferenceNo

Same TaskId

=====

EXAMPLE

APR-2026-000001

TaskId = X

Cycle 1:

Submitted

→ Director requests changes

Reason:

"Please attach quotation."

Cycle 2:

Resubmitted

→ Director requests changes

Reason:

"Please update budget."

Cycle 3:

Resubmitted

→ Director approves

Final:

ReferenceNo = APR-2026-000001

TaskId = X

CurrentCycleNo = 3

Status = Approved

ALL THREE CYCLES MUST REMAIN IN DATABASE.

=====

APIs

POST

/api/approvals/{taskId}/submit

POST

/api/approvals/{taskId}/approve

POST
/api/approvals/{taskId}/reject

POST
/api/approvals/{taskId}/request-changes

POST
/api/approvals/{taskId}/resubmit

GET
/api/approvals/{taskId}/cycles

GET
/api/approvals/{taskId}/history

=====

APPROVAL

=====

Approve only when request is:

PendingApproval

Record:

- Decision
- Actor
- Timestamp
- Comment if supplied
- Cycle
- History

Set:

WorkflowStatus = Approved

=====

REJECT

=====

Reject only when:

PendingApproval

Require a reason/comment.

Set:

WorkflowStatus = Rejected

Record history.

=====

REQUEST CHANGES

=====

Only allowed from:

PendingApproval

Require change reason.

Set current cycle:

ChangesRequested

Do NOT create a new cycle yet.

Do NOT create new TaskId.

Do NOT create new ReferenceNo.

=====

RESUBMIT

=====

Only allowed from:

ChangesRequested

Create:

new CycleNo = previous CycleNo + 1

Keep:

same TaskId

same ReferenceNo

Set:

WorkflowStatus = PendingApproval

Record:

Resubmitted

=====

INVALID TRANSITIONS

=====

Reject:

Approved → Approve

Approved → Reject

Approved → Request Changes

Rejected → Approve

Rejected → Request Changes

Draft → Approve

Draft → Reject

PendingApproval → Resubmit

ChangesRequested → Approve

etc.

Return clear validation errors.

=====

TRANSACTION

=====

Submit/Approve/Reject/Request Changes/Resubmit must update:

- Request
- Cycle
- History

consistently.

Use transaction handling.

=====

TESTS

=====

Test:

1. Draft → Submit.
2. Submit creates Cycle 1.
3. Approve.
4. Reject.
5. Request Changes.
6. Resubmit creates Cycle 2.
7. TaskId remains unchanged.
8. ReferenceNo remains unchanged.
9. Cycle 1 remains unchanged.
10. Cycle 2 created.
11. Second rework creates Cycle 3.
12. All cycles remain available.
13. Invalid transitions fail.
14. Comments/reasons persist.
15. History contains every lifecycle event.

Build and run all tests.

STEP 5 — Documents / Uploads

STEP 5 — APPROVAL DOCUMENT MANAGEMENT

Implement document management ONLY for Approval Management.

Reuse the existing backend file-upload/storage infrastructure discovered earlier.

Do not create a second unrelated file-storage system if an existing reusable mechanism exists.

=====
APIs
=====

POST

/api/approvals/{taskId}/documents

GET

/api/approvals/{taskId}/documents

GET

/api/approvals/{taskId}/documents/{documentId}

DELETE

/api/approvals/{taskId}/documents/{documentId}

=====
DOCUMENT DATA
=====

Store:

- DocumentId
- ApprovalRequestId
- TaskId
- CycleId
- FileName
- OriginalFileName
- StorageReference/FilePath
- ContentType
- FileSize
- DocumentType
- UploadedBy
- UploadedAt
- IsDeleted if required

=====

CYCLE LINKING

=====

Documents must be traceable to the cycle in which they were uploaded.

Example:

Task X
Reference APR-2026-000001

Cycle 1:
quotation.pdf
invoice.pdf

Cycle 2:
updated-quotation.pdf

All remain associated with the same TaskId.

Do NOT create a new TaskId or ReferenceNo for Cycle 2.

=====

HISTORY

=====

Record:

DocumentUploaded
DocumentRemoved

in ApprovalHistory.

=====

RULES

=====

Follow existing:

- File size limits
- Allowed file types
- Storage paths
- Security rules
- Naming conventions

Do not invent a new security mechanism.

Do not expose unsafe filesystem paths.

=====

TESTS

=====

Test:

1. Upload.
2. List documents.
3. Retrieve/preview.
4. Correct TaskId.
5. Correct CycleId.
6. Multiple documents.
7. Document remains in Cycle 1 history.
8. Cycle 2 documents correctly linked.
9. Delete where permitted.
10. History generated.

Build and test.

STEP 6 — Reminders + Escalations

STEP 6 — APPROVAL REMINDER AND ESCALATION BACKEND

Implement Reminder and Escalation functionality ONLY for Approval Management.

IMPORTANT:

The current frontend prototype used:

"Send Reminder"

"Escalate"

These were previously simulated.

Now create the backend foundation for real Approval Management reminder/escalation records.

Do NOT integrate email, WhatsApp, SMS, push notifications, or external messaging unless an existing notification infrastructure is already available and specifically reusable.

```
=====
REMINDER API
=====
```

```
POST
/api/approvals/{taskId}/reminders
```

```
GET
/api/approvals/{taskId}/reminders
```

```
=====
REMINDER DATA
=====
```

Store:

- ApprovalRequestId
- TaskId
- CycleId
- ReminderType
- SentTo
- SentAt
- TriggeredBy
- Comment
- Status

Every reminder must be associated with the current approval cycle.

```
=====
ESCALATION API
=====
```

```
POST
/api/approvals/{taskId}/escalations
```

```
GET
/api/approvals/{taskId}/escalations
```

```
=====
ESCALATION DATA
=====
```

Store:

- ApprovalRequestId
- TaskId
- CycleId
- EscalationLevel
- EscalatedFrom
- EscalatedTo
- Reason
- EscalatedAt
- Status
- ResolvedAt

=====

DUE DATE LOGIC

=====

Expose enough information for frontend to display:

Normal
Due Soon
Due Today
Overdue
Escalated

RequiredApprovalDate belongs to the active approval cycle.

Do not automatically send real messages unless existing notification infrastructure is intentionally connected.

=====

OVERDUE RULE

=====

An escalation should only be allowed according to the business rule for overdue approval.

Do not allow escalation of a completed/approved request.

Do not create a new TaskId.

Do not create a new ReferenceNo.

Escalation remains under the same:

TaskId
ReferenceNo
CycleId

=====

HISTORY

=====

Record:

ReminderSent
Escalated

in ApprovalHistory.

=====

TESTS

=====

Test:

1. Reminder creation.
2. Reminder history.
3. Reminder linked to TaskId.
4. Reminder linked to current CycleId.
5. Escalation creation.
6. Escalation history.
7. Escalation linked to same TaskId.
8. Escalation does not create new TaskId.
9. Escalation does not create new ReferenceNo.
10. Cannot escalate an approved request.
11. Correct overdue behavior.

Build and test.

STEP 7 — Complete List + Details + Dashboard Data

STEP 7 — APPROVAL MANAGEMENT QUERY APIs

Implement the read/query APIs required by the Approval Management frontend.

STRICT SCOPE:

Approval Management only.

=====

LIST API

=====

GET

/api/approvals

Support:

- Search
- Status filter
- Priority filter
- Approver filter
- RequestedBy filter
- Department filter
- Date filtering
- Pagination if existing project convention supports it
- Sorting if existing project convention supports it

Return at minimum:

- TaskId
- ReferenceNo
- RequestTitle
- RequestedBy
- Type
- Priority
- Approver
- RequiredApprovalDate
- WorkflowStatus
- DueState
- CurrentCycleNo
- DocumentCount
- ReminderCount
- EscalationCount
- CreatedAt
- UpdatedAt

=====

DETAIL API

=====

GET

/api/approvals/{taskId}

Return:

Request details

+

Current cycle

+

All cycles

+

Documents

+
History
+
Reminders
+
Escalations
+
Due-date state

=====

CYCLES

=====

GET

/api/approvals/{taskId}/cycles

Return all cycles ordered:

Cycle 1
Cycle 2
Cycle 3
...

Do not return only the latest cycle.

=====

HISTORY

=====

GET

/api/approvals/{taskId}/history

Return complete chronological history.

Include:

- Created
- DraftSaved
- Submitted
- Viewed
- DocumentUploaded
- ReminderSent
- ChangesRequested
- Resubmitted
- Approved
- Rejected

- Escalated

=====
KPI DATA
=====

Provide the backend data required for:

- Total Requests
- Pending Approval
- Approved
- Rejected
- Overdue
- Escalated

If the existing backend convention supports a dedicated dashboard endpoint, implement:

GET
/api/approvals/dashboard

Otherwise provide these counts through the existing list/query pattern.

=====
IMPORTANT
=====

Do not count every rework cycle as a separate request.

Example:

One request:

APR-2026-000001

Cycle 1
Cycle 2
Cycle 3

This is:

1 request

not:

3 requests.

KPIs must operate on **ApprovalRequest**, not **ApprovalCycle**.

Build and test.

STEP 8 — Final Security, Concurrency, Transactions & Full Tests

This is the final verification prompt.

STEP 8 — FINAL APPROVAL MANAGEMENT BACKEND HARDENING AND VERIFICATION

Perform a complete review of the Approval Management backend implemented in Steps 1–7.

STRICT SCOPE:

Approval Management only.

Do not redesign unrelated modules.

=====
1. ID SAFETY
=====

Verify:

ReferenceNo is:

- Backend generated
- Unique
- Sequential according to design
- Concurrent-safe
- Immutable

TaskId is:

- Backend generated
- Globally unique
- Immutable
- Never regenerated

Verify that:

Draft update:
same IDs

Submit:

same IDs

Request Changes:
same IDs

Resubmit:
same IDs

Escalation:
same IDs

Approval:
same IDs

Rejection:
same IDs

=====

2. REWORK SAFETY

=====

Verify:

Request:
Cycle 1

Request Changes:
Cycle 1 remains.

Resubmit:
Cycle 2 created.

Second Request Changes:
Cycle 2 remains.

Second Resubmit:
Cycle 3 created.

Final approval:
Cycle 3 approved.

All previous cycles remain intact.

ReferenceNo never changes.

TaskId never changes.

=====

3. DOCUMENT SAFETY

=====

Verify documents are correctly linked to:

ApprovalRequest

TaskId

CycleId

Historical documents remain traceable.

=====

4. HISTORY

=====

Verify every important event is recorded.

History must be append-only.

Do not overwrite previous events.

=====

5. WORKFLOW VALIDATION

=====

Verify only valid transitions are allowed.

Valid:

Draft → PendingApproval

PendingApproval → Approved

PendingApproval → Rejected

PendingApproval → ChangesRequested

ChangesRequested → PendingApproval

Invalid transitions must return appropriate errors.

=====

6. REMINDER

=====

Verify:

Reminder belongs to correct request.

Reminder belongs to correct TaskId.
Reminder belongs to current CycleId.

=====

7. ESCALATION

=====

Verify:

Escalation belongs to correct request.
Escalation belongs to same TaskId.
Escalation belongs to current CycleId.

Do not create new IDs.

=====

8. CONCURRENCY

=====

Test concurrent request creation.

Verify no duplicate:

ReferenceNo

Test concurrent resubmission.

Verify no duplicate:

CycleNo

Use database constraints and transactions where necessary.

=====

9. TRANSACTIONS

=====

Verify:

Submit
Approve
Reject
Request Changes
Resubmit
Escalate

cannot leave partially updated data.

Request state, cycle and history must remain consistent.

=====

10. AUTHORIZATION

=====

Use the existing authentication/authorization system.

Do not modify JWT/authentication architecture.

Ensure appropriate existing users can:

- Create requests
- View their requests
- Submit requests
- Approve where authorized
- Reject where authorized
- Request changes where authorized
- Resubmit where authorized
- Send reminders where authorized
- Escalate where authorized

Do not invent or modify roles unless absolutely required by the existing architecture.

=====

11. DATABASE

=====

Inspect the final migration.

Verify it does NOT:

- Drop unrelated tables.
- Modify unrelated columns.
- Change unrelated constraints.
- Change unrelated modules.

=====

12. API TESTING

=====

Test every endpoint:

POST /api/approvals

GET /api/approvals

GET /api/approvals/{taskId}

PUT/PATCH /api/approvals/{taskId}

POST /api/approvals/{taskId}/submit

POST /api/approvals/{taskId}/approve

POST /api/approvals/{taskId}/reject

POST /api/approvals/{taskId}/request-changes

POST /api/approvals/{taskId}/resubmit

GET /api/approvals/{taskId}/cycles

GET /api/approvals/{taskId}/history

POST /api/approvals/{taskId}/documents

GET /api/approvals/{taskId}/documents

GET /api/approvals/{taskId}/documents/{documentId}

DELETE /api/approvals/{taskId}/documents/{documentId}

POST /api/approvals/{taskId}/reminders

GET /api/approvals/{taskId}/reminders

POST /api/approvals/{taskId}/escalations

GET /api/approvals/{taskId}/escalations

=====

13. COMPLETE LIFECYCLE TEST

=====

Run this exact scenario:

Create Request

↓

Reference:

APR-2026-000001

TaskId:

X

↓

Cycle 1

↓

Submit

↓

Director requests changes

Reason:

"Please attach quotation."

↓

Cycle 1 = ChangesRequested

↓

Upload document

↓

Resubmit

↓

Cycle 2

↓

Director requests changes again

Reason:

"Please update budget."

↓

Cycle 2 = ChangesRequested

↓

Resubmit

↓

Cycle 3

↓

Send Reminder

↓

Escalate if overdue

↓

Director Approves

↓

Final:

ReferenceNo:
APR-2026-000001

TaskId:
X

CurrentCycleNo:
3

Status:
Approved

Verify that:

- Cycle 1 remains.
- Cycle 2 remains.
- Cycle 3 remains.
- All documents remain traceable.
- All reminders remain traceable.
- Escalation remains traceable.
- Complete history remains available.
- ReferenceNo never changed.
- TaskId never changed.

=====
14. TEST SUITE
=====

Run all existing backend tests.

Run all new Approval Management tests.

Test at minimum:

- Create
- Draft
- Update
- Submit
- Approve
- Reject
- Request Changes
- Resubmit
- Multiple rework cycles
- ID immutability
- Reference uniqueness
- Task uniqueness
- Concurrent reference generation
- Concurrent cycle generation
- Documents
- Document history
- Reminder
- Escalation
- Due dates
- Overdue state
- Invalid transitions
- Authorization
- Transactions

=====

FINAL REPORT

=====

Return:

1. Backend files changed
2. Entities created
3. Tables created
4. Database indexes
5. Database constraints
6. Migration name
7. Controllers
8. Services
9. DTOs
10. APIs
11. Workflow states
12. ReferenceNo generation strategy
13. TaskId strategy

14. Cycle/rework strategy
15. Document strategy
16. Reminder strategy
17. Escalation strategy
18. Authorization behavior
19. Tests executed
20. Tests passed/failed
21. Build result
22. Any unresolved issue

IMPORTANT:

Do not modify anything outside Approval Management unless absolutely required by an existing shared infrastructure dependency.

If an unrelated change appears necessary, STOP and report it before making that change.